

EXPERIMENT NO. 1

Aim:

Perform tokenization (Whitespace, Punctuation-based, Treebank, Tweet, MWE) using NLTK library. Use porter stemmer and snowball stemmer for stemming. Use any technique for lemmatization.

Theory:

What is Tokenization?

Tokenization is a process of converting raw data into a useful data string. Tokenization is used in NLP for splitting paragraphs and sentences into smaller chunks that can be more easily assigned meaning.

Tokenization can be done to either at word level or sentence level. If the text is split into words it is called word tokenization and the separation done for sentences is called sentence tokenization.

Why is Tokenization required?

In tokenization process unstructured data and natural language text is broken into chunks of information that can be understood by machine.

Tokenization converts an unstructured string (text document) into a numerical data structure suitable for machine learning. This allows the machines to understand each of the words by themselves, as well as how they function in the larger text. This is especially important for larger amounts of text as it allows the machine to count the frequencies of certain words as well as where they frequently appear.

Tokenization is the first crucial step of the NLP process as it converts sentences into understandable bits of data for the program to work with. Without a proper / correct tokenization, the NLP process can quickly devolve into a chaotic task.

Challenges of Tokenization

- Dealing with segment words when spaces or punctuation marks define the boundaries of the word. For example: donâ€™t
- Dealing with symbols that might change the meaning of the word significantly. For example: ₹100 vs 100
- Contractions such as ‘you’re’ and ‘I’m’ should be properly broken down into their respective parts. An improper tokenization of the sentence can lead to misunderstandings later in the NLP process.
- In languages like English or French we can separate words by using white spaces, or punctuation marks to define the boundary of the sentences. But this method is not applicable for symbol based languages like Chinese, Japanese, Korean Thai, Hindi, Urdu, Tamil, and others. Hence a common tokenization tool that combines all languages is needed.

Types of Tokenization

1. Word Tokenization

- Most common way of tokenization, uses natural breaks, like pauses in speech or spaces in text, and splits the data into its respective words using delimiters (characters like ' ' or ' ' or " " ").
- Word tokenization's accuracy is based on the vocabulary it is trained with. Unknown words or Out Of Vocabulary (OOV) words cannot be tokenized.

2. White Space Tokenization

- Simplest technique, Uses white spaces as basis of splitting.
- Works well for languages in which the white space breaks apart the sentence into meaningful words.

3. Rule Based Tokenization

- Uses a set of rules that are created for the specific problem.
- Rules are usually based on grammar for particular language or problem.

4. Regular Expression Tokenizer

- Type of Rule based tokenizer
- Uses regular expression to control the tokenization of text into tokens.

5. Penn Treebank Tokenizer

- Penn Treebank is a corpus maintained by the University of Pennsylvania containing over four million and eight hundred thousand annotated words in it, all corrected by humans
- Uses regular expressions to tokenize text as in Penn Treebank

Perform tokenization (Whitespace, Punctuation-based, Treebank, Tweet, Multi-Word Expression - MWE) using NLTK library.

In [20]:

```
!pip install nltk
```

Looking in indexes: <https://pypi.org/simple>, <https://us-python.pkg.dev/colab-wheels/public/simple/>

Requirement already satisfied: nltk in /usr/local/lib/python3.8/dist-packages (3.7)

Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.8/dist-packages (from nltk) (2022.6.2)

Requirement already satisfied: tqdm in /usr/local/lib/python3.8/dist-packages (from nltk) (4.64.1)

Requirement already satisfied: joblib in /usr/local/lib/python3.8/dist-packages (from nltk) (1.2.0)

Requirement already satisfied: click in /usr/local/lib/python3.8/dist-packages (from nltk) (7.1.2)

Whitespace based Tokenization

Syntax : tokenize.WhitespaceTokenizer()

Return : Return the tokens from a string

In [21]:

```
# import WhitespaceTokenizer() method from nltk
from nltk.tokenize import WhitespaceTokenizer

# Create a reference variable for Class WhitespaceTokenizer
wt = WhitespaceTokenizer()

# Create a string input
text = "Welcome to the I2IT-NLP Page. \n Good Morning \t"
print("\nOriginal string:")
print(text)
# Use tokenize method
tokenized_text = wt.tokenize(text)
print("\nSplitting using whitespece into separate tokens:")
print(tokenized_text)
```

Original string:

Welcome to the I2IT-NLP Page.

Good Morning

Splitting using whitespece into separate tokens:

['Welcome', 'to', 'the', 'I2IT-NLP', 'Page.', 'Good', 'Morning']

Punctuation-based tokenizer

In [22]:

```
from nltk.tokenize import WordPunctTokenizer
text = "Welcome to the I2IT-NLP Page. \n Good Morning \t"
print("\nOriginal string:")
print(text)
result = WordPunctTokenizer().tokenize(text)
print("\nSplit all punctuation into separate tokens:")
print(result)
```

Original string:

Welcome to the I2IT-NLP Page.

Good Morning

Split all punctuation into separate tokens:

```
['Welcome', 'to', 'the', 'I2IT', '-', 'NLP', 'Page', '.', 'Good', 'Morning']
```

Treebank Tokenizer

In [23]:

```
from nltk.tokenize import TreebankWordTokenizer
```

```
tokenizer = TreebankWordTokenizer()
tokenizer.tokenize(text)
```

Out[23]:

```
['Welcome', 'to', 'the', 'I2IT-NLP', 'Page.', 'Good', 'Morning']
```

Tweet Tokenizer

When we want to apply tokenization in text data like tweets, the tokenizers mentioned above can't produce practical tokens. Through this issue, NLTK has a rule based tokenizer special for tweets. We can split emojis into different words if we need them for tasks like sentiment analysis.

In [24]:

```
from nltk.tokenize import TweetTokenizer
```

```
tweet_tokenize = TweetTokenizer()
sample_tweet = "Who is your favourite cryptocurrency influencer? 🐼 Tag them below! 🐼"
print(tweet_tokenize.tokenize(sample_tweet))
```

```
['Who', 'is', 'your', 'favourite', 'cryptocurrency', 'influencer', '?', '🐼', '🐼', 'Tag', 'them', 'below', '!', '🐼']
```

Multi-Word Expression Tokenizer

A MWETokenizer takes a string which has already been divided into tokens and retokenizes it, merging multi-word expressions into single tokens, using a lexicon of MWEs.

The multi-word expression tokenizer is a rule-based, “add-on” tokenizer offered by NLTK. Once the text has been tokenized by a tokenizer of choice, some tokens can be re-grouped into multi-word expressions.

For example, the name Steven Spielberg is combined into a single token instead of being broken into two tokens. This tokenizer is very flexible since it is agnostic of the base tokenizer that was used to generate the tokens.

In [25]:

```
# import MWETokenizer() method from nltk
from nltk.tokenize import MWETokenizer
#from nltk.tokenize import word_tokenize
```

```
# Create a reference variable for Class MWETokenizer
```

```
tokenizer = MWETokenizer([('a', 'little'), ('a', 'little', 'bit'), ('a', 'lot')])
```

```
tokenizer.add_mwe(['in', 'spite', 'of'])
```

```
tokenizer.tokenize('In a little or a little bit or a lot in spite of'.split())
```

Out[25]:

```
['In', 'a_little', 'or', 'a_little_bit', 'or', 'a_lot', 'in_spite_of']
```

In [26]:

```
from nltk.tokenize import MWETokenizer
```

```
tokenizer = MWETokenizer()
```

```
tokenizer.add_mwe(['Steven', 'Spielberg'])
```

```
tokenizer.tokenize('Steven Spielberg is an American writer producer director '.split())
```

Out[26]:

```
['Steven_Spielberg', 'is', 'an', 'American', 'writer', 'producer', 'director']
```

Stemming

Stemming is a process of reducing inflectional words to their root form. It maps the word to a same stem even if the stem is not a valid word in the language.

Why is stemming required?

English language has several variants of a single term. The presence of these variances in a text corpus results in data redundancy when developing NLP or machine learning models. Such models may be ineffective.

To build a robust model, it is essential to normalize text by removing repetition and transforming words to their base form through stemming.

Types of Stemmers in NLTK

- Porter Stemmer
- Snowball Stemmer
- Lancaster Stemmer
- Regexp Stemmer

Porter Stemmer

It is a type of stemmer which is mainly known for Data Mining and Information Retrieval. As its applications are limited to the English language only. It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes, it is also majorly known for its simplicity and speed. The advantage is, it produces the best output from other stemmers and has less error rate.

In [27]:

```
import nltk
from nltk.stem import PorterStemmer

ps = PorterStemmer()
example_words = ["connector", "connection", "connects", "connecting", "connected"]
```

#Next, we can easily stem by doing something like:

```
for w in example_words:
    print(ps.stem(w))
```

```
connector
connect
connect
connect
connect
connect
```

Snowball Stemmer

In [28]:

```
from nltk.stem import SnowballStemmer
snowball = SnowballStemmer(language='english')
words = ['generous', 'generate', 'generously', 'generation']
for word in words:
    print(word, "--->", snowball.stem(word))
```

```
generous ---> generous
generate ---> generat
generously ---> generous
generation ---> generat
```

Lancaster Stemmer

In [29]:

```
from nltk.stem import LancasterStemmer
lancaster = LancasterStemmer()
words = ['eating', 'eats', 'eaten', 'puts', 'putting']
for word in words:
    print(word, "--->", lancaster.stem(word))
```

```
eating ---> eat
eats ---> eat
eaten ---> eat
puts ---> put
putting ---> put
```

Regex Stemmer

```
from nltk.stem import RegexpStemmer
regex = RegexpStemmer('ing|s|e|able', min=4)
words = ['mass', 'was', 'bee', 'computer', 'advisable']
for word in words:
    print(word, "--->", regex.stem(word))
```

```
mass ---> mas
was ---> was
bee ---> bee
computer ---> computer
advisable ---> advis
```

Porter Vs Snowball Vs Lancaster Vs Regex Stemmers

```
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer, RegexpStemmer
porter = PorterStemmer()
lancaster = LancasterStemmer()
snowball = SnowballStemmer(language='english')
regex = RegexpStemmer('ing|s|e|able', min=4)
word_list = ['generous', 'generate', 'generously', 'generation']
print("{0:20}{1:20}{2:20}{3:30}{4:40}".format("Word", "Porter Stemmer", "Snowball Stemmer", "Lancaster Stemmer", "Regex Stemmer"))
for word in word_list:
    print("{0:20}{1:20}{2:20}{3:30}{4:40}".format(word, porter.stem(word), snowball.stem(word), lancaster.stem(word), regex.stem(word)))
```

Word	Porter Stemmer	Snowball Stemmer	Lancaster Stemmer	Regex Stemmer
generous	gener	generous	gen	generou
generate	gener	generat	gen	generat
generously	gener	generous	gen	generously
generation	gener	generat	gen	generation

Lemmatization

Use any technique for lemmatization.

Lemmatization, unlike stemming reduces the inflected words properly ensuring that the root word belongs to the language.

Lemmatization is the grouping together of different forms of the same word. In search queries, lemmatization allows end users to query any version of a base word and get relevant results. Because search engine algorithms use lemmatization, the user is free to query any inflectional

form of a word and get relevant results. For example, if the user queries the plural form of a word (e.g., routers), the search engine knows to also return relevant content that uses the singular form of the same word (router).

Lemmatization is extremely important because it is far more accurate than stemming. This brings great value when working with a chatbot where it is crucial to understand the meaning of a user's messages.

The major disadvantage to lemmatization algorithms, however, is that they are much slower than stemming algorithms.

Using NLTK Library for Lemmatization

In [34]:

```
import nltk
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
```

Out[34]:

True

In [35]:

```
import nltk
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()
```

```
sentence = "He was running and eating at same time. He has bad habit of swimming after playing long hours in the Sun."
```

```
punctuations="?!.,;"
```

```
sentence_words = nltk.word_tokenize(sentence)
```

```
for word in sentence_words:
```

```
    if word in punctuations:
```

```
        sentence_words.remove(word)
```

```
sentence_words
```

```
print("{0:20}{1:20}".format("Word","Lemma"))
```

```
for word in sentence_words:
```

```
    print("{0:20}{1:20}".format(word,wordnet_lemmatizer.lemmatize(word)))
```


Word	Lemma
He	He
was	wa
running	running
and	and
eating	eating
at	at
same	same
time	time
He	He
has	ha
bad	bad
habit	habit
of	of
swimming	swimming
after	after
playing	playing
long	long
hours	hour
in	in
the	the
Sun	Sun